



FACULTY OF INFORMATION TECHNOLOGY

BIT1123 – Object Oriented Programming

FINAL PROJECT - GROUP (40%)

**LifeFlow: A Java-Based Blood Donation and
Inventory Coordination System**

SEMESTER 202605

LECTURER NAME: NAZMIRUL IZZAD BIN NASIR

PREPARED BY:

No.	Name	Student ID	Program	Passport Number	Class Code
1	ALI MOHAMMED RIYADH YASEEN FAREA	202501010286	BCSSE	14036858	BIT1123
2	Jiha Omar	202501010623	BCSSE	N01622707	BIT1123
3	KISRANE AMINE	202505010489	BCSSE	307528754	BIT1123
4	GHAITH ALDAIN GAMIL HAMID MELHI QASEM	202501010660	BCSSE	11772197	BIT1123
5	Omar Algaafari	202505010463	BCSSE	10646149	BIT1123

GitHub Repository: <https://github.com/moha-r/LifeFlow>

Table of Contents

<i>Executive Summary</i>	<i>3</i>
<i>1. Introduction and SDG Background</i>	<i>4</i>
1.1 Project overview and scope.....	4
1.2 Relationship to Sustainable Development Goal 3	4
1.3 Global and Malaysian context.....	5
<i>2. Problem Statement</i>	<i>6</i>
2.1 The coordination problem.....	6
2.2 Integrity, persistence, and role boundaries	7
2.3 Problem boundaries.....	7
<i>3. System Objectives and Requirements</i>	<i>8</i>
3.1 Objectives.....	8
3.2 Role-based use cases	9
3.3 Why the system is more than CRUD	10
<i>4. OOP Design and UML</i>	<i>11</i>
4.1 Architecture and object responsibilities.....	11
4.2 Classes, objects, and encapsulation	13
4.3 Getters, setters, and controlled mutation.....	13
4.4 Inheritance and runtime polymorphism	14
4.5 Abstraction, interfaces, and generics.....	15
4.6 UML interpretation	16
<i>5. Implementation Details</i>	<i>17</i>
5.1 Technology stack and startup.....	17
5.2 GUI structure.....	18
5.3 Eligibility and inventory states.....	18

5.4 Request ordering and matching.....	19
5.5 Appointments and volunteer coordination.....	20
5.6 Transaction integrity.....	20
5.7 File handling and persistence	21
5.8 Reporting and code quality.....	22
6. Testing and Sample Outputs	23
6.1 Test architecture and verified baseline	23
6.2 Representative behavioural tests	24
6.3 Interpretation of sample outputs	25
7. Discussion and Limitations	30
7.1 Strengths and design trade-offs	30
7.2 Security, medical, and operational limitations	30
7.3 Testing and maintainability limitations	31
8. Conclusion	32
8.1 Conclusion.....	32
References.....	33
Appendix A : Representative Test Matrix	34
Appendix B: Team Contribution Matrix.....	34

Executive Summary

LifeFlow is a Java Swing desktop application designed as an educational simulation of blood-donation, inventory, request, and appointment coordination. The project addresses Sustainable Development Goal 3 by modelling a practical operational problem: a blood service must keep time-limited stock visible, distinguish emergency demand from routine demand, coordinate donors and hospitals, and preserve a reliable record of every state change. The application is intentionally narrower than a clinical blood-bank platform. It does not screen blood, perform laboratory crossmatching, recommend transfusions, or replace professional medical judgement.

The implemented system supports three roles. Administrators manage donor profiles, blood units, hospital accounts, requests, appointments, matching, and reports. Hospitals place and monitor blood requests and review volunteer appointments. Donors view their eligibility result, donation history, compatible urgent needs, and appointments. The processing model goes beyond create, read, update, and delete operations. It orders requests by subtype-defined priority, request date, and identifier; evaluates educational eligibility rules; selects stock by exact or compatible blood type; allocates units by first-expire-first-out; protects partially reserved requests; and treats multi-entity changes as transactions.

The design demonstrates encapsulation, inheritance, runtime polymorphism, abstraction, generics, collections, and file handling. A versioned JSON envelope, SHA-256 checksum, atomic replacement, two backup generations, migration logic, and a single-instance file lock protect the main state file. The verified baseline contains 73 production Java files and 35 test Java files. A clean Maven run on 24 August 2026 executed 229 Surefire test cases with zero failures, errors, skips, or compiler warning lines. The strongest evidence is behavioural: tests cover matching order, compatibility, rollback, corruption detection, migration, appointment ownership, reservation release, user-interface structure, and report escaping.

LifeFlow therefore meets the central technical intent of the assignment while remaining an educational local application. Its main limitations are plain-text credentials, fixed administrator credentials, one large controller, local single-user storage, no measured code-coverage percentage, and simplified medical rules.

1. Introduction and SDG Background

1.1 Project overview and scope

LifeFlow was developed to demonstrate how object-oriented software can represent a real operational process rather than a collection of unrelated data-entry screens. Its subject is the coordination of blood donors, stored blood units, hospital demand, matching decisions, and donation appointments. These activities are connected. A donor profile determines the blood type and the eligibility context for a donation; a recorded donation creates a blood unit with a defined simulated shelf life; a hospital request creates demand; and the matching workflow either fulfils that demand, reserves part of the required stock, or reports that the available inventory is insufficient. The system also records the outcome so that later views and reports agree with the transaction that occurred.

The scope is deliberately educational. LifeFlow is a desktop simulation written for an Object Oriented Programming course, not a clinical information system. It treats each stored unit as a simplified whole-blood unit, uses a compact ABO/Rh compatibility matrix, and applies configurable classroom rules for age, weight, waiting period, and shelf life. These rules are useful for demonstrating decisions, state transitions, and validation, but they omit screening, infectious-disease testing, component processing, crossmatching, prescribing, consent, adverse-event reporting, and other controls required in real transfusion services. This boundary is stated throughout the interface and this report so that software behaviour is not confused with clinical guidance.

The application is also local by design. Swing provides the graphical interface, while the application layer coordinates a copyable domain state and JSON persistence. No network server, national database, or external patient record is contacted. The result is a self-contained system that can be demonstrated in a classroom, inspected in source control, and tested deterministically without claiming deployment readiness.

1.2 Relationship to Sustainable Development Goal 3

The selected goal is Sustainable Development Goal 3, which concerns healthy lives and well-being. The most defensible connection is operational support for access to health services, especially the need for timely and sufficient blood supplies. Target 3.8 refers to access to quality essential health-care services, while Target 3.d addresses capacity for health-risk management (United Nations, n.d.). LifeFlow does not measure progress against either target.

Instead, it uses blood-supply coordination as a concrete software problem through which students can model limited resources, urgent demand, traceable decisions, and reliable records.

The connection is supported by the real characteristics of blood services. The World Health Organization reports that timely access to safe blood remains unequal and recommends coordinated national systems, effective donor management, quality-assured screening, and consistent supply networks (World Health Organization, 2026a). LifeFlow represents only a small operational slice of that much larger system. It can track donor and hospital accounts, model stock availability, sort urgent requests, and preserve the result of a simulated allocation. It cannot establish that a donation is safe or that a unit is suitable for a particular patient.

This distinction prevents an exaggerated sustainability claim. A classroom application cannot improve national availability by itself, and it has no evidence of real-world health outcomes. Its value lies in demonstrating how information systems can make operational states visible and decisions reproducible. The project therefore aligns with SDG 3 as an educational design exercise: the problem domain is relevant to health-service availability, and the software illustrates some coordination principles without presenting itself as a public-health intervention.

1.3 Global and Malaysian context

Blood availability depends on regular collection, safe processing, storage, and distribution. WHO estimates that approximately 120.4 million blood donations are collected worldwide and documents substantial differences in donation rates between income groups. It also identifies a stable base of regular, voluntary, unpaid donors as an important foundation for an adequate supply (World Health Organization, 2026a). The 2025 global status report further treats access to quality-assured blood products as part of resilient health systems and universal health coverage, while documenting continuing differences in governance, safety, and availability across 168 reporting countries (World Health Organization, 2026b).

Malaysia provides a useful local context because the Ministry of Health publishes blood-donation information through KKMNOW. The dashboard describes a stable supply as important for a functioning public health system and explains that its near-real-time indicators are derived from the Blood Bank Information System version 2 used by 22 principal collection sites (Ministry of Health Malaysia, 2026a). The related dataset reports

national daily donations by major blood group. Its caveat is equally important: the integrated sites account for roughly 80 per cent of donations, so the dataset is representative rather than complete (Ministry of Health Malaysia, 2026b).

LifeFlow does not import, reproduce, or claim compatibility with BBISv2. The Malaysian sources establish that donation activity varies over time and that blood group is an operational variable, but they are not specifications for this project. The application uses fictional records and a separate local schema. This separation allows the report to explain why the problem matters without suggesting that a student simulation has the coverage, safety controls, or institutional authority of the Ministry of Health system.

2. Problem Statement

2.1 The coordination problem

The problem addressed by LifeFlow is the coordination of several changing records that must remain consistent. Demand is not constant. A hospital may submit a routine request and later submit an emergency request with a different blood type and quantity. Stock also changes. A unit may be scheduled, available, reserved, used, discarded, or considered expired on the evaluation date. A donor may be eligible today but become deferred after a recorded donation. An appointment may be booked, completed, cancelled, or become stale when its date passes. Treating each table independently would produce contradictions, such as a fulfilled request with no consumed units or a reserved unit with no pending request.

Urgency introduces a decision problem. Emergency requests should normally be considered before regular requests, but priority alone does not guarantee that the required stock exists. LifeFlow therefore separates the next pending request from the next fulfilable request. MatchingService orders requests by priority, then request date, then identifier, while inventory filtering determines whether a complete allocation is possible. This avoids a queue in which an unfulfillable emergency request permanently blocks every later request. The rule is deterministic, so the same state and matching mode produce the same choice.

Compatibility and expiry add further constraints. In exact mode, the unit blood type must equal the request type. In compatible mode, the request type calls `BloodType.canReceiveFrom` to evaluate the simplified ABO/Rh matrix. Candidate units are sorted by expiry date and then identifier, implementing first-expire-first-out. This reduces simulated wastage and makes the selection explainable. If fewer units are available than the

outstanding quantity, the normal matching operation changes nothing. Volunteer appointments are the exception: completed volunteer donations may reserve units against a pending request until the request becomes fully covered.

2.2 Integrity, persistence, and role boundaries

Operational correctness includes failure handling. A request fulfilment affects at least the request status, several unit statuses, a fulfilment record, the audit log, and the state revision. Saving only part of this change would make later reports unreliable. LifeFlowController therefore constructs a complete candidate snapshot, validates it, saves it, and only then publishes it as the active state. A persistence exception leaves the prior state and revision unchanged. This save-before-publish sequence is exercised by tests that inject failing stores and compare the entire state before and after the attempted operation.

Role boundaries address a different risk. Administrators have system-wide management functions. A hospital can create and manage its own requests and complete appointments assigned to it, but it cannot manage another hospital's records. A donor portal is restricted to the account's linked donor profile, its own appointments, its donation history, and compatible urgent needs. Ownership checks are therefore part of the service operations, not merely a visual convention in the interface.

Persistence must also survive ordinary desktop risks. The main JSON store uses a format-versioned envelope, a checksum over canonical content, atomic file replacement, current and previous backups, explicit recovery, and a directory lock. These controls do not make the application a database, but they reduce the chance of accepting a partially written or silently modified snapshot. The account stores are simpler and are therefore identified as a limitation. The main store's checksum is also schema-sensitive: changing serialized fields without a migration can invalidate older envelopes even when the data was valid for the earlier build.

2.3 Problem boundaries

LifeFlow solves an information-management simulation, not the medical problem of deciding whether blood should be collected or transfused. The application does not verify identity, obtain consent, perform donor interviews, test infectious markers, separate components, determine patient compatibility, or record clinical outcomes. Its eligibility result is a classroom decision based on age, weight, waiting period, and dates already entered into the system. Its blood-type compatibility matrix is a simplified software rule. Any real

deployment would require clinical governance, validated workflows, security engineering, privacy controls, regulatory review, and integration with laboratory and hospital systems.

These exclusions are not hidden shortcomings. They define a responsible project boundary and make the implemented objectives testable. The project can be evaluated on object-oriented design, meaningful processing, consistency, persistence, graphical output, and automated testing without pretending to satisfy requirements that were never implemented.

3. System Objectives and Requirements

3.1 Objectives

The first objective is to manage the core entities needed by the simulation: donors, blood units, hospital accounts, requests, appointments, fulfilments, and audit entries. Management includes validated creation and controlled updates, but it also includes cross-entity rules. A blood unit must reference an existing donor and use the donor's blood type. A fulfilment must reference an existing request and the exact units committed to it. A booked appointment must reference a valid donor and hospital, and a linked request must remain pending.

The second objective is to provide distinct workflows for administrators, hospitals, and donors. The administrator requires an operational overview and system-wide management pages. The hospital requires a focused request form, a list of its own requests, fulfilment details, and appointments. The donor requires an eligibility explanation, donation history, appointment controls, and a list of compatible urgent needs. These interfaces share the same application state but expose different actions and ownership rules.

The third objective is to implement explainable processing. Eligibility produces a reason and next eligible date rather than a bare Boolean. Request ordering uses explicit priority, date, and identifier rules. Matching supports exact and compatible modes, uses FEFO within the eligible stock, and reports insufficient stock without partial mutation. Appointment completion records the donation and can reserve or fulfil a linked request. The fourth objective is persistence reliability: the application should save a complete state locally, detect corrupted envelopes, recover from verified backups, migrate the supported legacy format, and prevent two processes from writing the same directory.

The final objective is verifiability. Important behaviour must be observable in the GUI, operational reports, or tests. The project does not claim a test-coverage percentage because no

coverage tool is configured. Instead, it presents a fresh build result and representative tests whose arrangements, actions, and expected outcomes can be discussed directly.

Table 1. Functional requirements and implementation evidence

ID	Requirement	Role	Input	Processing	Output	Code evidence
FR-01	Manage donor profiles	Administrator	Validated donor details	Create or controlled update; calculate eligibility context	Donor table and status	LifeFlowController.addDonor/updateDonor
FR-02	Record donation units	Administrator/Hospital	Donor and donation date	Eligibility, expiry calculation and state validation	New unit and history	addBloodUnit/completeDonationAppointment
FR-03	Manage hospital accounts	Administrator	Name, username and password	Uniqueness validation and immediate save	Donation-centre account	HospitalRegistry.register/update/remove
FR-04	Place a blood request	Administrator/Hospital	Type, quantity and kind	Create subtype and bind hospital ownership	Pending request	LifeFlowController.addRequest
FR-05	Order pending demand	Administrator	Pending requests	Priority, date and identifier comparator	Next pending/fulfillable request	MatchingService.findNextPending/findNextFulfillable
FR-06	Match stock	Administrator	Request, date and mode	Availability, exact/compatible filter and FEFO	Unit allocation or shortage	MatchingService.match
FR-07	Fulfil a request	Administrator	Selected request and units	All-or-nothing candidate commit	Used units and fulfilment record	processSpecificRequest
FR-08	Decline a request	Administrator/Hospital owner	Pending request and reason	Terminal guard, cancellation and release	Cancelled request and audit	declineRequest
FR-09	Book a donation	Donor	Hospital, date and optional need	Eligibility, compatibility and one-active rule	Booked appointment	bookDonationAppointment
FR-10	Complete an appointment	Hospital owner	Booked appointment and date	Create unit; reserve or fulfil linked request	Completed appointment and inventory update	completeDonationAppointment
FR-11	Persist and recover	System/Administrator	Complete state snapshot	Validate, checksum, atomic write and backup	Verified JSON or explicit recovery	JsonLifeFlowStore.save/load/restoreLatestBackup
FR-12	Export operational reports	Administrator	Current state and date	Escape and format CSV/HTML	Six report outputs	CsvReportExporter/HtmlReportExporter

3.2 Role-based use cases

The administrator begins with an operations overview showing donor count, available units, pending demand, emergency demand, and appointments. From the eight workspace pages, the administrator can register and update donor profiles, record eligible donations, correct dates on unused units, discard stock, create or edit pending requests, decline requests with a reason, inspect the matching queue, process a request, review appointments, maintain donation-centre accounts, and export reports. The dashboard's next-action card combines

priority and availability so that the operator sees whether the leading request can be processed.

A hospital signs in with its own account. The hospital dashboard shows only its requests and appointments. The request form accepts blood type, quantity, and kind, while the controller assigns a stable identifier and binds the request to the hospital identifier. The hospital can cancel its own pending request and inspect the unit identifiers attached to a fulfilled request. For donation appointments, the hospital can record a completed donation only when it owns the appointment and the appointment is valid for completion.

A donor signs in through an account linked to one donor profile. The portal explains current eligibility, shows blood type and donation history, displays appointments, and lists pending urgent needs compatible with the donor's type. A donor can book one active appointment, volunteer for a compatible request, cancel an owned booking, update limited profile information, and change the local password. If the administrator removes the donor profile, the portal displays a removal message and disables profile-dependent actions. These role-specific paths demonstrate that the GUI is not simply an administrator table reused for every user.

3.3 Why the system is more than CRUD

CRUD operations change one record at a time. LifeFlow's central operations instead evaluate conditions and move several records through coordinated states. Recording a donation checks the donor, calculates an expiry date, creates a unit, updates effective donation history, validates the complete candidate state, and saves the result. Processing a request selects a polymorphic request, calculates the remaining quantity after reservations, filters stock by matching mode and date, sorts it by FEFO, changes unit status, changes request status, writes a fulfilment record, appends audit information, and commits one revision.

The appointment workflow is similarly stateful. Booking checks ownership, future date, one-active-booking policy, donor eligibility on the appointment date, request compatibility, and request status. Completion creates a new unit. For a linked request, that unit may be reserved while coverage is incomplete or used when the completed donation reaches the required quantity. Declining the request cancels linked bookings and releases reserved units.

Automatic stale cleanup cancels only records that satisfy the configured age and commitment rules. These transitions depend on loops, comparisons, collections, and conditional decisions, satisfying the assignment's requirement for meaningful processing and simulation logic.

4. OOP Design and UML

4.1 Architecture and object responsibilities

LifeFlow uses a layered desktop architecture. Main creates account registries, resolves the storage directory, shows the login dialog on Swing's event-dispatch thread, and routes the successful result to the administrator, hospital, or donor frame. The UI layer owns visual composition and user interaction. It does not write JSON directly. Frames and panels call LifeFlowController or the appropriate account registry, then refresh their views from a new state snapshot.

The application layer centres on LifeFlowController. It coordinates DonationPolicy, BloodInventory, MatchingService, DataValidator, and LifeFlowStore. Domain objects represent state and local behaviour, while LifeFlowState groups the complete persistent snapshot. The persistence boundary is the LifeFlowStore interface; JsonLifeFlowStore is its file-based implementation. This separation is visible in tests, which replace the real store with an in-memory or deliberately failing implementation without changing controller code.

The architecture is not a strict enterprise framework. Some services are static exporters, and LifeFlowController is larger than ideal. Nevertheless, the direction of dependency is clear enough for the assignment: Swing depends on the service layer, services depend on the domain model and persistence abstraction, and the JSON implementation depends on serializable domain data. Figure 1 summarises that arrangement.

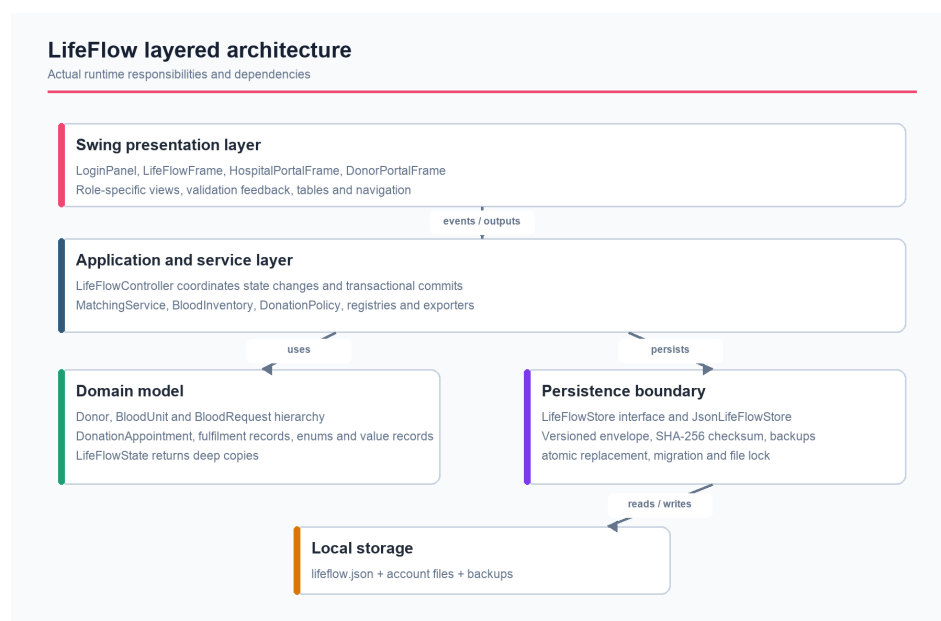
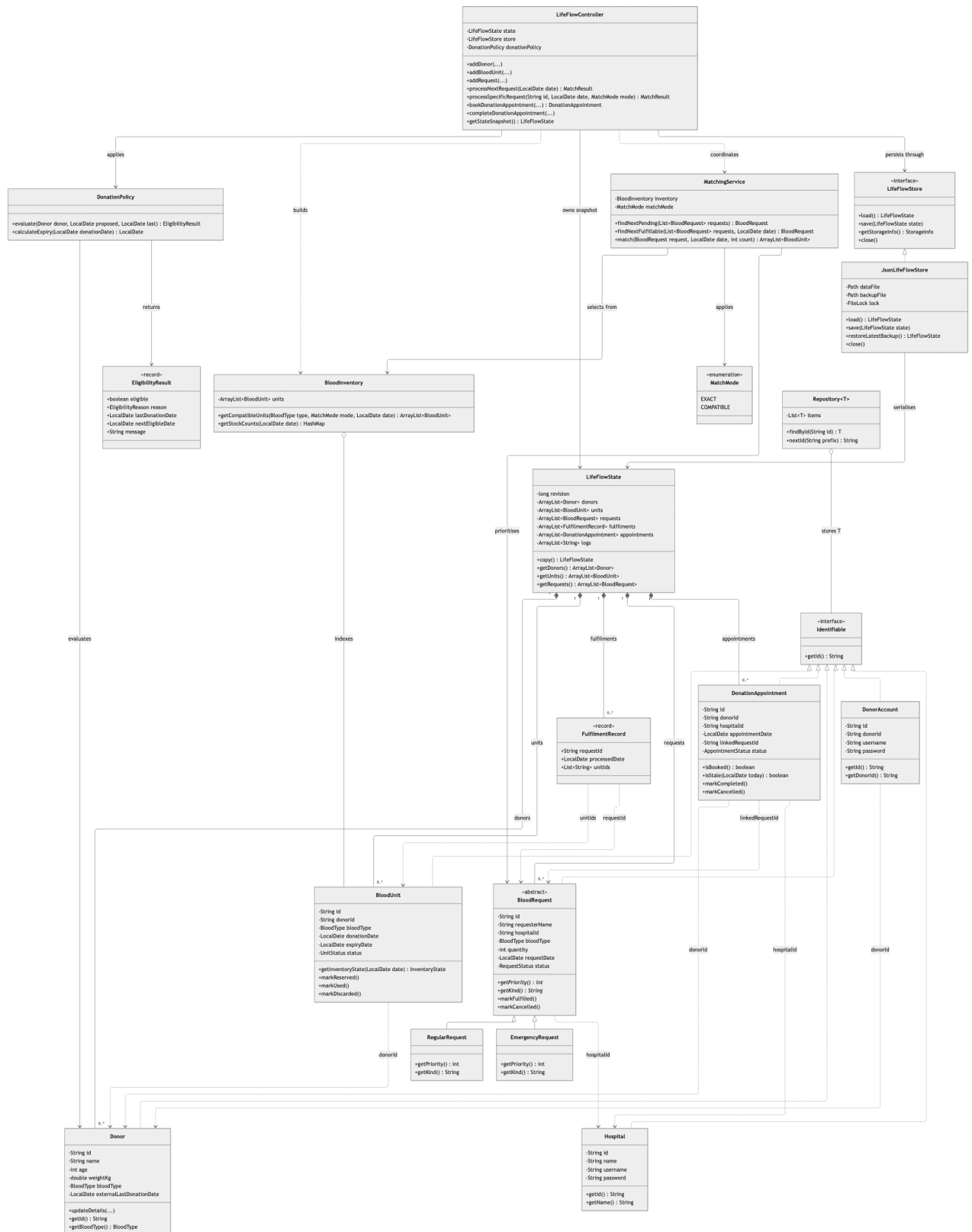


Figure 1. Layered architecture and principal dependencies



4.2 Classes, objects, and encapsulation

The domain contains more than the minimum six to eight classes, but the relevant evidence is not the count alone. Donor represents one donor profile. BloodUnit represents one simulated whole-blood unit and derives its visible inventory state from dates and UnitStatus.

BloodRequest is an abstract request with shared fields and controlled transitions.

DonationAppointment models ownership, linkage, date, and status. FulfilmentRecord and EligibilityResult are records that carry immutable result data. Hospital and DonorAccount represent authentication and role context. LifeFlowState groups the aggregate without exposing its internal lists directly.

Fields in these classes are private. Read access is provided through methods such as `getBloodType`, `getRequestDate`, and `getStatus`. Mutations with domain meaning use named methods instead of arbitrary field setters. `Donor.updateDetails` changes a validated group of profile fields. `BloodRequest.updatePendingDetails` is limited to a pending request, while `markFulfilled` and `markCancelled` express terminal transitions. `BloodUnit` exposes `markReserved`, `markUsed`, `markAvailable`, and `markDiscarded`. These methods make the allowed change visible in code and allow the controller to guard completed or cancelled records.

`LifeFlowState` adds a second encapsulation boundary. Every collection getter returns a deep copy of mutable domain objects, and `copy` creates a fully separate snapshot. A caller therefore cannot change the published state by mutating a returned list. The controller must capture one set of lists, apply all changes to those lists, create a candidate state, and commit that candidate. This design prevented aliasing but introduced an implementation hazard: calling a getter twice produces two unrelated copies, so a change made to the first list is lost if the second list is passed to commit. The corrected appointment-completion path follows the single-capture rule.

4.3 Getters, setters, and controlled mutation

The assignment specifically asks for getters and setters. `LifeFlow` clearly supplies getters but intentionally avoids generic `setX` methods for state that has business rules. A literal `setStatus` method on `BloodRequest` could allow `PENDING` to become `FULFILLED` without assigning any units. A `setDonationDate` method on `BloodUnit` could create an expiry mismatch or alter a used unit. The controller therefore exposes use-case methods and the model exposes controlled transition methods.

This is stronger encapsulation from a design perspective, but the difference from the rubric wording should be acknowledged. The implementation meets the purpose of setters, which is controlled write access, through `updateDetails`, `updatePendingDetails`, `correctDates`, `markFulfilled`, `markCancelled`, `markReserved`, `markAvailable`, `markUsed`, and `markDiscarded`. It does not meet a narrow interpretation that expects method names beginning with `set`. The report does not invent such methods. During assessment, the team should explain that named domain operations prevent invalid intermediate states and point to the validation and rollback tests that support this choice.

Read access is also selective. The controller supplies snapshots and computed counts rather than returning its internal state reference. Registries return copies of account lists. Password accessors exist because the current local authentication implementation compares plain text, but this is a security limitation rather than a recommended pattern. Future work should replace reversible credential storage with password hashing and remove direct password exposure.

Table 2. Read access and controlled write operations

Object	Read access	Controlled write	Reason
Donor profile	<code>getName</code> , <code>getAge</code> , <code>getWeightKg</code> , <code>getBloodType</code>	<code>updateDetails</code>	Fields are validated as one profile update.
Blood request	<code>getStatus</code> , <code>getQuantity</code> , <code>getRequestDate</code>	<code>updatePendingDetails</code> ; <code>markFulfilled</code> ; <code>markCancelled</code>	Generic status assignment could bypass fulfilment rules.
Blood unit	<code>getDonationDate</code> , <code>getExpiryDate</code> , <code>getStatus</code>	<code>correctDates</code> ; <code>markReserved</code> ; <code>markUsed</code> ; <code>markAvailable</code> ; <code>markDiscarded</code>	Named transitions preserve inventory meaning.
Appointment	<code>getAppointmentDate</code> , <code>getStatus</code> , <code>isBooked</code>	<code>markCompleted</code> ; <code>markCancelled</code>	Only controller-validated terminal changes are exposed.
Application state	deep-copy collection getters	controller use-case methods	No caller receives the mutable published lists.

4.4 Inheritance and runtime polymorphism

`BloodRequest` defines the inheritance hierarchy required by the brief. It stores the common identifier, requester, hospital identifier, blood type, quantity, date, and status. It also declares abstract `getPriority` and `getKind` methods. `RegularRequest` and `EmergencyRequest` extend `BloodRequest` and override both methods. A regular request returns priority 1 and kind `REGULAR`, while an emergency request returns priority 2 and kind `EMERGENCY`.

Runtime polymorphism occurs because requests are stored and processed as `BloodRequest` references. `LifeFlowState` contains an `ArrayList` of `BloodRequest`, `MatchingService` accepts

List<BloodRequest>, and its comparator calls getPriority on each superclass reference. The method executed depends on the object's runtime subtype. The same mechanism supplies the kind displayed in tables and reports. No subtype test is needed to determine priority. A subtype check does appear when a deep copy must instantiate the correct concrete class, which is a separate construction concern rather than the dispatch mechanism.

This hierarchy is meaningful because emergency and regular requests share state and transitions but differ in behaviour used by the queue. It is also deliberately small. Creating many subclasses without behavioural differences would make the model harder to explain. Figure 3 isolates the inheritance and dispatch path so that the runtime relationship remains readable even when the complete class diagram is reduced to a page.

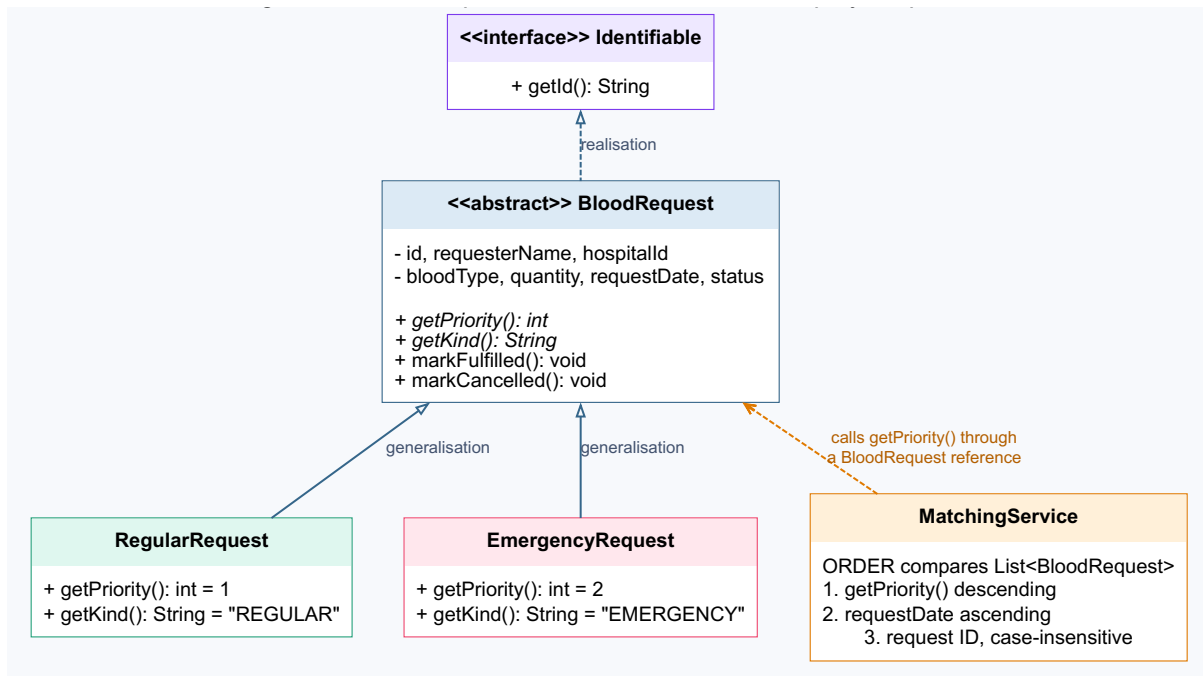


Figure 3. *BloodRequest inheritance and runtime method dispatch*

4.5 Abstraction, interfaces, and generics

Interfaces define several replaceable contracts. **Identifiable** supplies `getId` for domain objects that participate in generic lookup. **LifeFlowStore** abstracts loading, saving, storage information, and closing; the production implementation is **JsonLifeFlowStore**, while tests use small in-memory and failing stores. **StateObserver** represents notification after a published state change. **ReportExporter** declares an export contract, although the current CSV and HTML exporters use static methods and do not implement it. That inconsistency is documented rather than hidden.

LifeFlowException is an abstract typed exception base. DuplicateIdException, EntityNotFoundException, ValidationException, EligibilityException, ImmutableRecordException, and InsufficientStockException add context appropriate to different failures. The UI can catch the shared type for user feedback while tests can assert the specific subtype and its structured fields. This is a useful form of abstraction because it separates expected domain failures from arbitrary runtime exceptions.

Repository<T extends Identifiable> demonstrates bounded generics. The type parameter allows the same lookup and monotonic identifier logic to work with donors, units, requests, appointments, and accounts while requiring an identifier at compile time. Collections are chosen by use. ArrayList preserves a predictable iteration order for stored entities. HashMap provides keyed aggregation such as stock counts and validation indexes. Map and Set detect duplicates and track relationships. Streams and comparators express filtering and deterministic ordering. The design uses generics and collections as part of the processing model, not merely to satisfy a checklist.

4.6 UML interpretation

The complete UML diagram shows the main domain, services, persistence contract, and controller dependencies. Composition from LifeFlowState to donors, units, requests, fulfilments, appointments, and logs represents snapshot ownership. Realisation arrows connect Identifiable and LifeFlowStore to their implementations. Generalisation connects BloodRequest to RegularRequest and EmergencyRequest. Dependencies based on identifiers are shown separately from direct object ownership because records store donorId, hospitalId, requestId, and unitIds rather than nested mutable objects.

The complete diagram is dense because it provides traceability across several assignment requirements. It should be read as an orientation map, not the only explanation. The architecture view and inheritance detail enlarge the two relationships most likely to be discussed during assessment. The transaction figure in Section 5 then explains a dynamic collaboration that a static class diagram cannot show. Together, these figures preserve readable labels while retaining the full model for reference.

5. Implementation Details

5.1 Technology stack and startup

The build targets Java 17 and packages a shaded executable JAR with `lifeflow.Main` as its entry point. Swing supplies the desktop interface. Maven manages compilation, testing, and packaging. The current POM specifies Jackson 2.22.0 for JSON processing, JUnit Jupiter 5.12.2 for automated tests, Maven Compiler Plugin 3.13.0, Maven Surefire Plugin 3.5.2, and Maven Shade Plugin 3.6.0. These versions were read from the build file rather than copied from the README.

Main starts the graphical workflow with `SwingUtilities.invokeLater`, keeping Swing construction on the event-dispatch thread. It resolves the data directory through `StoragePaths`, loads hospital and donor account files, creates registries and the donor signup service, and displays `LoginDialog`. A `LoginResult` record identifies an administrator, hospital, or donor session. `SessionSwitcher` closes the current frame and routes to another role or back to login while ensuring that each controller releases its store lock. A `CountDownLatch` keeps the process alive until the final session exits.

The login panel provides three role tabs. Administrator authentication uses the fixed local credentials `admin` and `admin123`. Hospital and donor credentials are read from separate JSON account stores. This is acceptable for a self-contained classroom demonstration but not for production because the passwords are stored and compared as plain text. The report therefore treats authentication as role routing, not as a secure identity system.

Table 3. Verified technology stack

Category	Technology	Version	Evidence
Language	Java	17	<code>pom.xml</code> <code>maven.compiler.release</code>
Desktop UI	Swing	Java 17 platform	<code>javax.swing</code> imports
Build	Maven Wrapper	Project wrapper	<code>mvnw</code> and <code>pom.xml</code>
JSON	Jackson Databind	2.22.0	<code>pom.xml</code> property <code>jackson.version</code>
Testing	JUnit Jupiter	5.12.2	<code>pom.xml</code> property <code>junit.version</code>
Compilation	Maven Compiler Plugin	3.13.0	<code>pom.xml</code>
Test runner	Maven Surefire Plugin	3.5.2	<code>pom.xml</code>
Executable JAR	Maven Shade Plugin	3.6.0	<code>pom.xml</code>

5.2 GUI structure

The administrator frame contains eight pages grouped in a compact sidebar: Overview, Inventory, Requests, Matching, Donors, Appointments, Donation Centers, and Reports. The overview combines summary metrics, a next-action card, blood-group stock, a recall list, and a request queue. It provides quick navigation to donor, unit, request, and matching actions. Each management page uses a bounded central content area, consistent controls, validation messages, and read-only table models for displayed data.

The hospital portal uses a task-first layout. Its upper summary shows open, emergency, fulfilled, and scheduled counts. The main workspace places the blood-request form next to the hospital's own request table, followed by its appointments. The donor portal presents an eligibility status chip, blood type, last donation, donation count, appointments, donation history, and compatible urgent needs. These portals are scrollable at smaller window sizes and restrict actions based on ownership and current state.

Validation occurs at more than one layer. The UI prevents obvious empty or malformed input and displays typed exception messages. Service and model validation remain authoritative because tests and other callers can bypass the GUI. This avoids a common desktop error in which invalid state is blocked only by a particular form. The screenshots in Section 6 were rendered from the actual Swing components using fictional records; local storage paths are excluded from the report images.

5.3 Eligibility and inventory states

DonationPolicy applies four educational settings: age from 18 through 60, minimum weight of 45 kilograms, a three-month waiting period, and a 35-day unit shelf life. evaluate returns an EligibilityResult containing the Boolean outcome, a reason enum, the effective last donation date, the next eligible date, and an explanatory message. The effective last date considers both an external date recorded on the donor profile and internal donation units. calculateExpiry adds the configured shelf-life days to a donation date.

These values are application constants, not a complete statement of Malaysian donor eligibility. Actual acceptance involves more questions, screening, medical history, haemoglobin and infection testing, and professional procedures that LifeFlow does not implement. The Ministry of Health dashboard also describes different storage periods for real components, including red cells, platelets, and plasma (Ministry of Health Malaysia, 2026a).

LifeFlow's single 35-day whole-unit model must therefore remain labelled as an educational simplification.

BloodUnit separates stored status from date-dependent inventory state. UnitStatus records AVAILABLE, RESERVED, USED, or DISCARDED. getInventoryState first respects terminal and reserved statuses, then reports SCHEDULED if the donation date is later than the evaluation date, EXPIRED if the expiry date has passed, or AVAILABLE otherwise. This method lets reports distinguish a physically stored status from its operational meaning on a particular date. Date corrections are allowed only for unused stock, and validation checks donor identity, blood type, date order, and maximum simulated shelf life.

5.4 Request ordering and matching

MatchingService defines a deterministic request order. Higher getPriority values come first, so emergency requests precede regular requests. Ties are resolved by the earlier request date and then by identifier. findNextPending returns the leading pending request.

findNextFulfillable walks the ordered list and returns the first request for which enough units can be selected. This distinction allows a lower-priority but fulfilable request to proceed when a higher-priority request lacks stock, a behaviour verified by MatchingConsistencyTest and ControllerReliabilityTest.

BloodInventory filters out used, reserved, discarded, scheduled, and expired units. Exact mode requires equal blood types. Compatible mode calls requestType.canReceiveFrom(unitType) using the simulated ABO/Rh matrix. The resulting candidates are ordered by expiry date and identifier, which implements FEFO and makes ties reproducible. The normal processSpecificRequest operation calculates any quantity already reserved, requests the outstanding count, and either obtains the entire remaining allocation or throws InsufficientStockException. It does not partially consume general stock.

When the candidate allocation succeeds, units are marked used, the request is marked fulfilled, a FulfilmentRecord stores the request identifier, processing date, and exact unit identifiers, and an audit entry is added. A request dated after the matching date is rejected. Completed and cancelled requests are immutable across editing, processing, decline, and unit-related paths. These guards are necessary because a visual disabled button is not enough to protect the model.

5.5 Appointments and volunteer coordination

DonationAppointment links a donor, hospital, date, optional request, and status. Booking requires an existing donor and hospital, a valid future date, eligibility on the proposed date, and no other active appointment for the donor. A linked volunteer request must be pending and compatible with the donor's blood type. The controller generates appointment identifiers using the maximum existing numeric suffix plus one, avoiding loops that can repeat occupied identifiers after deletions.

Cancellation is restricted to the owning donor while the appointment is booked. Completion is restricted to the owning hospital and cannot occur before the appointment date. A valid completion creates a blood unit and marks the appointment completed in one transaction. If the appointment is linked to a request, the new unit is reserved when the accumulated committed quantity remains below the request quantity. When the final required unit arrives, all committed units are marked used, the request becomes fulfilled, and the fulfilment record contains the complete unit list.

Linked records also respond to request changes. Declining a request cancels booked volunteer appointments and releases its reserved units to available stock. Fulfilling a request through general matching cancels redundant volunteer bookings. Automatic stale-request cleanup avoids cancelling requests that already hold committed units. Past booked appointments are considered stale for the one-active-booking rule, allowing the donor to book again. These paths explain why appointments are part of the domain model rather than a separate calendar table.

5.6 Transaction integrity

Every multi-record update follows a candidate-state pattern. The controller captures one set of deep-copy lists from the current LifeFlowState. It performs the requested mutation on those captured objects, constructs a candidate with an incremented revision, and calls DataValidator on the complete candidate. Validation checks duplicate identifiers, required text, date ranges, donor-unit blood-type consistency, donation spacing, request validity, fulfilment quantities, unit status, appointment linkage, and active-booking counts.

The candidate is then passed to LifeFlowStore.save. JsonLifeFlowStore serializes and writes the snapshot. Only after save returns successfully does the controller assign the candidate to its state field and notify observers. If validation or persistence fails, the candidate is

abandoned. Because the published state was never mutated, the application retains the previous revision, statuses, unit list, fulfilments, appointments, and log. Figure 4 shows this sequence.

This pattern is particularly important because LifeFlowState getters return deep copies. It prevents external mutation but requires discipline inside the controller. A mutation must be applied to the list that is later used to build the candidate. Fetching a second copy for commit would silently lose the change. The completeDonationAppointment implementation and its regression tests reflect this rule. The design has a performance cost because snapshots copy every entity, but the data volume of a classroom desktop application is small and the correctness benefit is easier to demonstrate.

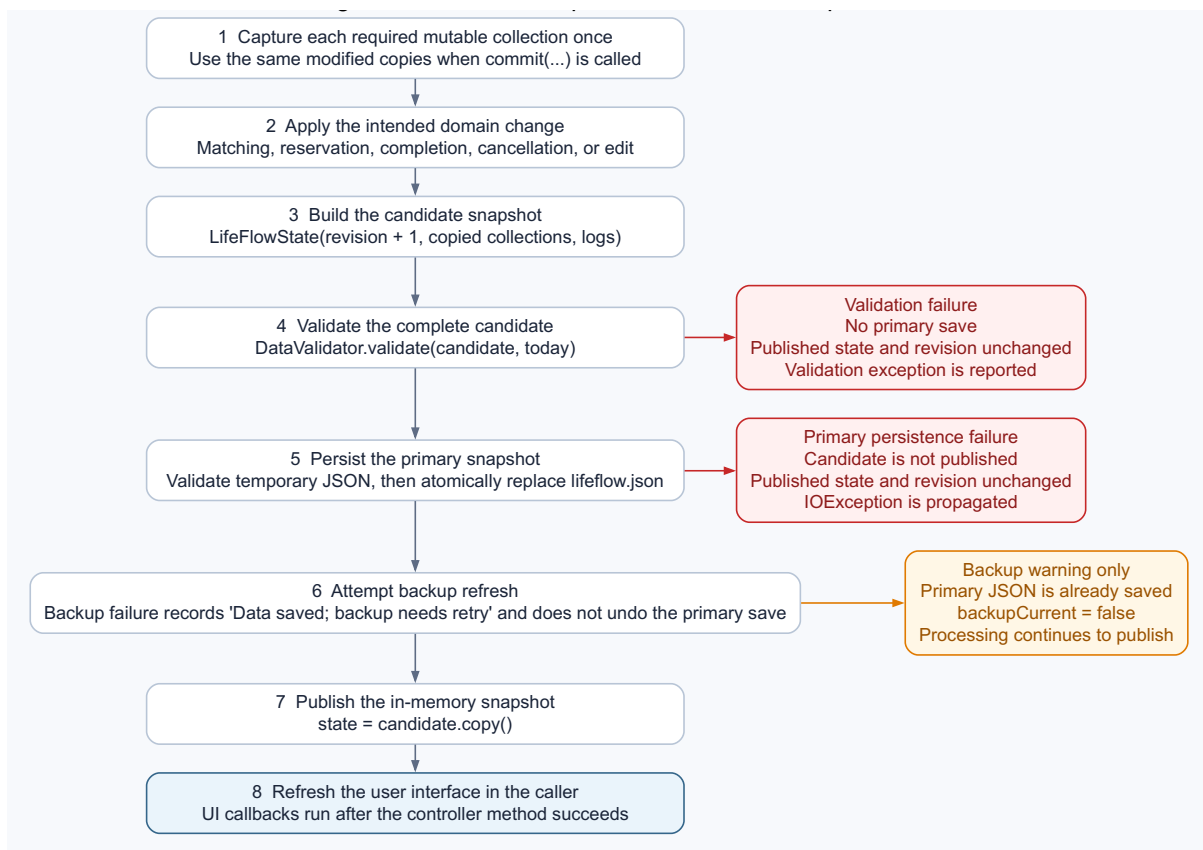


Figure 4. Save-before-publish transaction sequence

5.7 File handling and persistence

JsonLifeFlowStore writes the main state to lifeflow.json inside the resolved data directory. The version 2 envelope contains the data payload, format version, revision, and SHA-256 checksum. Jackson is configured for indented output, sorted map entries, and alphabetically sorted properties. The checksum is calculated over a canonical serialization of the payload,

format version, and revision. On load, the store verifies the envelope before constructing a state and passes the result through `DataValidator`.

Saving uses a temporary file and atomic replacement when the file system supports it. After the primary file is established, the store updates a current backup and retains a previous backup generation. Backup failure is reported in `StorageInfo` without discarding a successful primary save. A damaged or missing primary does not trigger silent recovery; the store marks recovery as required, and `restoreLatestBackup` explicitly selects a verified current or previous backup. A copy of a corrupt primary is retained in a recovery directory before replacement.

The store supports migration from format version 1 to version 2. The migration removes duplicated legacy donation-date representation while preserving a legacy date as external history when no unit represents it. An invalid migration leaves the original file untouched and reports the failure accurately. A `FileChannel` lock on `lifeflow.lock` prevents a second process from opening the same directory. These mechanisms satisfy the file-I/O requirement at a level beyond plain serialization.

The checksum also creates a compatibility obligation. Adding or removing a serialized DTO field changes the canonical content expected by a newer build. Older valid envelopes can then fail checksum verification unless the change is accompanied by a migration or a reserialization procedure. Persistence schema changes must therefore be planned, versioned, and tested. The separate hospital and donor account stores do not yet have the same envelope, backup, migration, and checksum design, so their resilience is lower.

5.8 Reporting and code quality

LifeFlow exports five CSV views: inventory, donors, requests, appointments, and audit. The inventory report includes operational state on the selected date. The donor report includes eligibility information. The request report includes kind and priority, while the appointment report includes status and linked request. The audit report combines fulfilment and log information. `CsvReportExporter` escapes commas and quotation marks so that names and messages remain valid CSV fields.

`HtmlReportExporter` creates an operational summary with counts, stock by blood type, donor eligibility, requests, appointments, and escaped text. It is a system output, not the academic project report. Tests insert commas and markup into sample data to verify CSV quoting and HTML escaping. `ReportExporter` exists as an abstraction but the concrete exporters currently

expose static methods rather than implementing it. Aligning the interface and implementations is a sensible future refactor.

The source is divided into model, service, persistence, and ui packages with a small root entry point. Classes and methods generally use domain-specific names, typed exceptions, enums, and records. Javadoc explains non-obvious classes and important behaviour, while focused comments document migration and failure-handling decisions. The clean build produced zero compiler warning lines. Remaining quality concerns are architectural rather than compiler defects: LifeFlowController has many responsibilities, account security is weak, and deep copying trades performance for isolation.

6. Testing and Sample Outputs

6.1 Test architecture and verified baseline

The automated suite combines current JUnit 5 tests with a legacy manual suite. Classes ending in Test use JUnit Jupiter annotations and are discovered directly by Surefire. Older classes ending in Tests contain assertion helpers and manual run methods. AllTests calls those legacy runners, and one JUnit bridge named LegacySuiteTest invokes AllTests so that the old assertions remain part of the Maven result. The figure of 229 is therefore the number of Surefire test cases, including the single bridge case; it should not be described as 229 separately authored JUnit methods.

Tests use fixed Clock instances when date-sensitive behaviour must be deterministic and @TempDir when file operations need isolated directories. In-memory LifeFlowStore implementations support service tests without disk. Failing stores throw IOException at a controlled point so that tests can compare the published state and revision before and after failure. UI tests construct Swing components, find named controls, and inspect tables, labels, dimensions, scroll panes, and enabled states. They are structural and behavioural component tests rather than full end-to-end automation of every rendered workflow.

A fresh ./mvnw clean test run on 24 August 2026 compiled 73 production Java files and 35 test Java files, executed 229 tests, and reported zero failures, zero errors, zero skipped tests, and zero compiler warning lines. The build completed successfully. No JaCoCo plugin or equivalent coverage tool is configured, so no percentage is claimed. The command output in Figure 12 is the evidence for the reported baseline.

Table 4. Fresh verification baseline

Measure	Verified result
Production Java files	73
Test Java files	35
Surefire tests	229
Failures / errors / skipped	0 / 0 / 0
Compiler warning lines	0
Build result	SUCCESS

6.2 Representative behavioural tests

`MatchingConsistencyTest.orderBreaksTiesByRequestDateThenId` verifies the final ordering rules after subtype priority.

`ControllerReliabilityTest.matchingUsesFefoAndPersistsTheAuditInOneSnapshot` verifies expiry ordering and an atomic audit result. Its

`insufficientStockDoesNotSaveOrChangeRevision` and

`failedMatchingSaveRollsBackUnitsRequestAndAuditTogether` methods check two distinct failure paths: a domain shortage and an I/O failure.

`JsonLifeFlowStoreTest.rejectsTamperedPrimaryAndRestoresVerifiedBackup` changes stored content and confirms that checksum verification prevents normal loading before explicit

recovery. `fallsBackToPreviousBackupAndMakesItCurrent` covers the second backup

generation. `refusesASecondProcessLockForTheSameDirectory` proves the single-instance

guard. `JsonMigrationTest.migratesVersionOneAndRemovesDuplicatedDonorDonationDate`

checks the supported format upgrade, while

`invalidLegacyHistoryLeavesOriginalFileUntouched` checks migration safety.

`AppointmentBookingTest` covers sequential identifiers, past dates, one active booking,

eligibility, compatibility, ownership, completion, round-trip persistence, partial reservations,

fulfilment from committed units, and release on decline. `RequestDeclineTest` verifies

terminal-state protection and required reasons. `DonorPortalUiTest` and

`HospitalAppointmentsUiTest` verify role-specific visibility and ownership.

`ReportExporterTest` checks eligibility output, request kind and priority, CSV escaping,

HTML escaping, appointment sections, and the exclusion of incomplete fulfilments. These

examples were selected because each can be explained through arrangement, action, and

expected state rather than only by quoting a test count.

Table 5. Representative automated tests

Behaviour	Test class and method	Evidence
Request priority and ties	MatchingConsistencyTest.orderBreaksTiesByRequestDateThenId	Priority, date, then identifier form a deterministic order.
FEFO allocation	ControllerReliabilityTest.matchingUsesFefoAndPersistsTheAuditInOneSnapshot	The earliest-expiring eligible unit is consumed and audit data shares one commit.
Insufficient stock	ControllerReliabilityTest.insufficientStockDoesNotSaveOrChangeRevision	No partial save or revision change occurs.
Save rollback	ControllerReliabilityTest.failedMatchingSaveRollsBackUnitsRequestAndAuditTogether	Injected I/O failure leaves the published snapshot unchanged.
Checksum corruption	JsonLifeFlowStoreTest.rejectsTamperedPrimaryAndRestoresVerifiedBackup	Tampering blocks normal load and recovery uses a verified backup.
Backup generation	JsonLifeFlowStoreTest.fallsBackToPreviousBackupAndMakesItCurrent	The previous verified backup remains recoverable.
Migration	JsonMigrationTest.migratesVersionOneAndRemovesDuplicatedDonorDonationDate	Version 1 data is converted to the current representation.
Appointment booking	AppointmentBookingTest.doubleBookingIsRejected	A donor cannot hold two active bookings.
Volunteer reservation	AppointmentBookingTest.volunteerDonationsReserveUnitsUntilRequestIsComplete	Committed units stay unavailable to unrelated matching.
Request decline	AppointmentBookingTest.decliningRequestReleasesReservedUnitsBackToStock	Cancellation restores committed stock.
Role UI	HospitalAppointmentsUiTest.hospitalSeesItsAppointmentsAndVolunteers	Hospital ownership and volunteer counts are visible.
Report safety	ReportExporterTest.summaryHtmlContainsMetricsAndEscapesMarkup	HTML output includes metrics without rendering injected markup.

6.3 Interpretation of sample outputs

The sample outputs use fictional donors, hospitals, accounts, requests, units, and appointments created in a temporary data directory. Figure 5 shows the three login roles. Figure 6 shows the administrator's operational overview with stock, pending demand, emergency demand, and appointments. Figure 7 combines the hospital request form with its request table; the fulfilled row demonstrates that the portal can display a completed result and assigned units. Figure 8 shows the donor's eligibility and appointment context, while Figure 9 shows the administrator's matching queue and mode selector. Figure 10 displays the cross-role appointment record.

Figure 11 is produced by `HtmlReportExporter` from the same fictional snapshot. It presents counts, inventory by blood type, and donor eligibility in a browser-readable form. Table 6 presents the inventory CSV excerpt as tabular evidence of file output. Figure 12 shows the clean Maven summary. These outputs demonstrate current behaviour only. They do not prove clinical correctness, security, accessibility, or deployment readiness, and they should not be treated as substitutes for a live classroom demonstration.

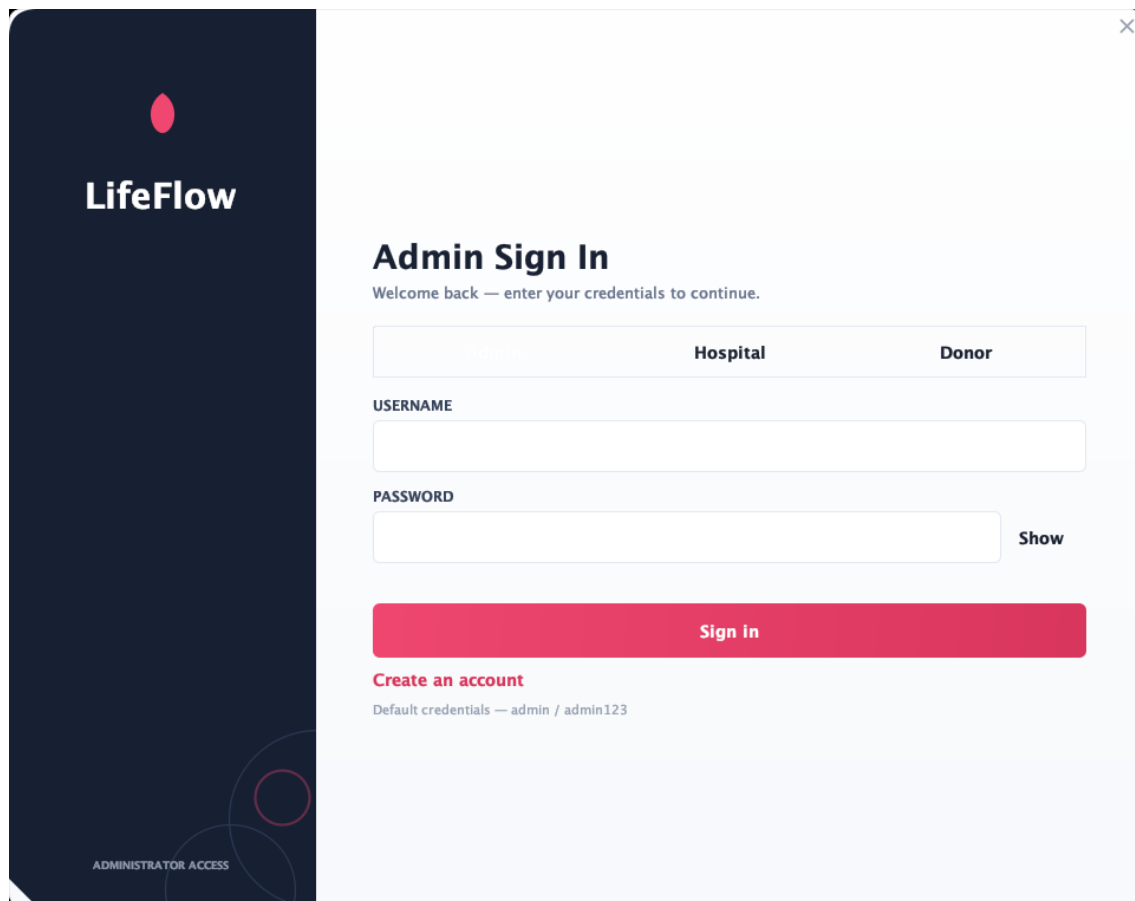


Figure 5. Login screen and three role choices

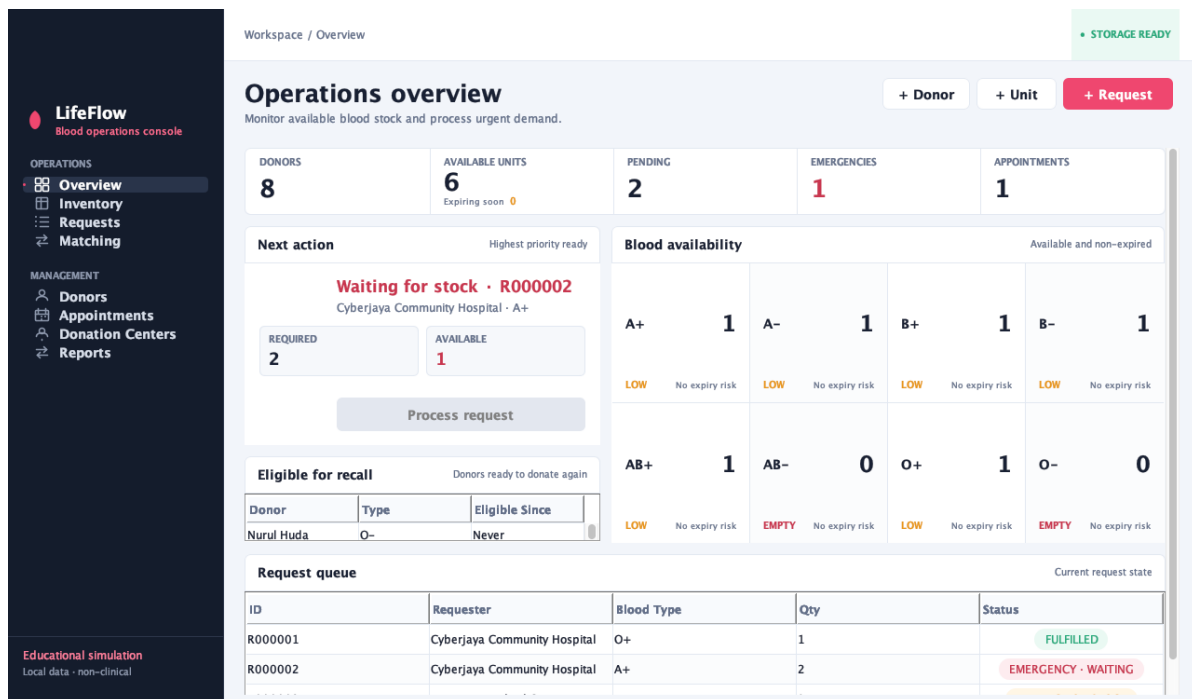


Figure 6. Administrator operations overview



LifeFlow

HOSPITAL PORTAL

Sign out

Hospital dashboard

Cyberjaya Community Hospital · Manage blood demand and donation appointments.

OPERATIONS SNAPSHOT

Cyberjaya Community...

Live activity at a glance

Open requests

1

Emergency open

1

Fulfilled

1

Create blood request

Choose the need and submit it for matching.

BLOOD TYPE

A+

QUANTITY (UNITS)

1

REQUEST KIND

Regular

Blood requests

2 request(s) · 1 pending

Request ID	Kind	Blood Type	Quantity	Date	Status	
R000001	REGULAR	O+	1	02 Aug 2026	FULFILLED	0
R000002	EMERGENCY	A+	2	24 Aug 2026	PENDING	1

Figure 7. Hospital request workspace and request status output



LifeFlow

DONOR PORTAL

Sign out

Donor dashboard

Your eligibility, appointments, and donation impact.

Edit profile

Change p...

DONATION STATUS

Hi, Nurul

ELIGIBLE TO DONATE

Donor is eligible on 2026-08-24.

Age 31 · 56.0 kg · Last donation: —

Blood type

O-

Last donation

—

Total donations

0

My appointments

1 appointment(s)

Cancel

Book a don...

Appointment	Hospital	Date	Status
A000001	Cyberjaya Community Hospital	27 Aug 2026	BOOKED

My donations

0 donation(s)

Urgent needs

Vol...

Figure 8. Donor eligibility and appointment workspace

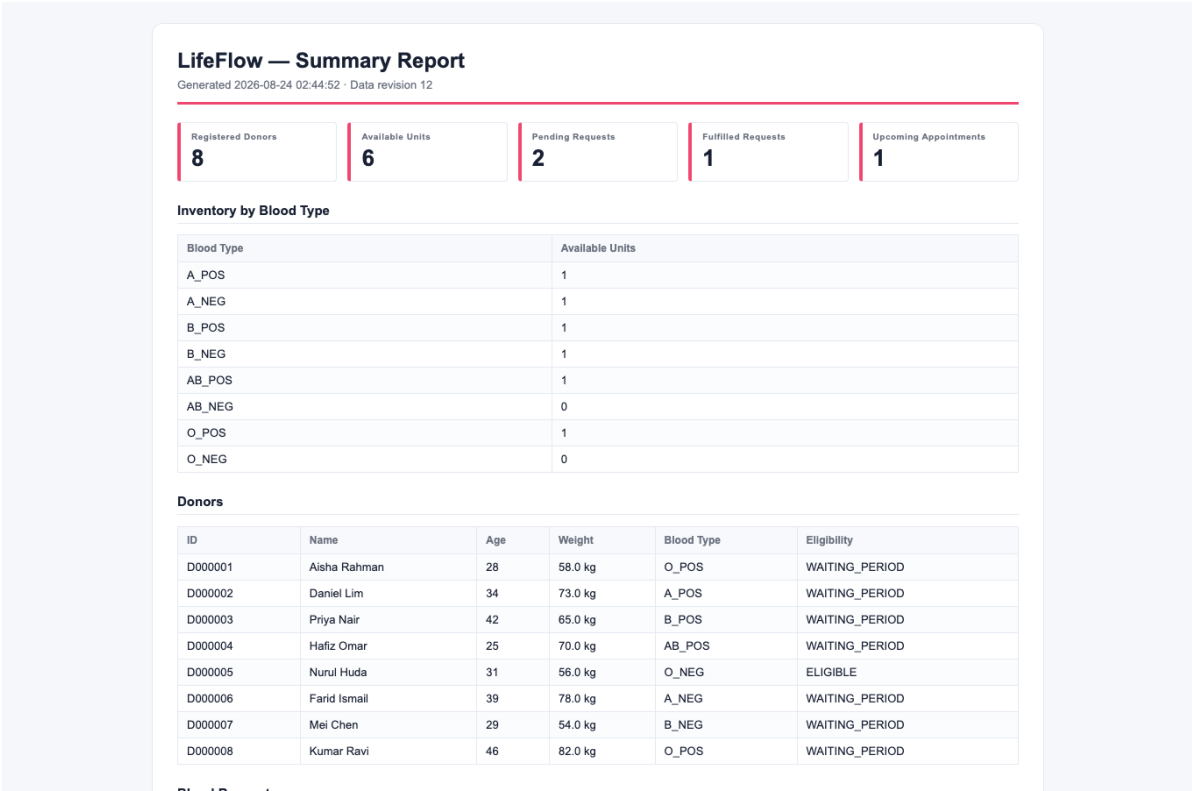


Figure 11. HTML operational summary generated from fictional data

```
$ ./mvnw clean test

[INFO] Compiling 73 source files with javac [debug release 17] to target/classes
[INFO] Compiling 35 source files with javac [debug release 17] to target/test-classes
[INFO] Tests run: 229, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 7.568 s
[INFO] Finished at: 2026-08-24T18:44:48+08:00
```

Figure 12. Clean Maven test result used as the verified baseline

Table 6. Inventory CSV sample output

Unit ID	Donor ID	Blood Type	Donation Date	Expiry Date	Status
U000001	D000001	O_POS	2026-08-10	2026-09-14	AVAILABLE
U000002	D000002	A_POS	2026-08-12	2026-09-16	AVAILABLE
U000003	D000003	B_POS	2026-08-15	2026-09-19	AVAILABLE
U000004	D000004	AB_POS	2026-08-18	2026-09-22	AVAILABLE
U000005	D000006	A_NEG	2026-08-17	2026-09-21	AVAILABLE
U000006	D000007	B_NEG	2026-08-19	2026-09-23	AVAILABLE

7. Discussion and Limitations

7.1 Strengths and design trade-offs

LifeFlow's strongest feature is the connection between object-oriented structure and observable behaviour. The BloodRequest hierarchy affects queue order at runtime. LifeFlowStore enables failure injection and a real JSON implementation. Repository uses a bounded type rather than raw collections. Typed domain methods prevent arbitrary status changes. These choices are not decorative examples; they participate in matching, persistence, and testing.

The simulation also goes beyond isolated CRUD screens. Request processing, volunteer reservations, stale cleanup, eligibility evaluation, date-dependent inventory state, and rollback require coordinated decisions across collections. Deterministic tie-breaking and FEFO make results explainable. A fulfilment record preserves the exact unit identifiers, allowing the hospital portal and reports to trace the outcome. The main persistence store adds checksum, backups, migration, recovery, and locking that are unusually thorough for a small course application.

Several strengths have costs. Deep copies protect the published state but allocate new objects and require the single-capture discipline. The controller offers one obvious transaction boundary but has grown large as features were added. Local JSON keeps deployment simple but limits concurrency and query capability. Component-level UI tests are fast and deterministic but do not prove every interaction on every operating system. These are reasonable educational trade-offs, provided they are presented honestly.

7.2 Security, medical, and operational limitations

Authentication is the most immediate security weakness. Administrator credentials are fixed in source code, and hospital and donor passwords are stored in plain text. There is no password hashing, logout, multi-factor authentication, secure reset flow, or authorization server. The data files are not encrypted. Anyone with operating-system access to the directory can read account and operational data. Fictional information is therefore appropriate for demonstrations, while real personal or medical data is not.

The medical model is simplified. Eligibility uses only age, weight, waiting period, and recorded dates. Blood units are not screened, separated into components, labelled with

laboratory results, or crossmatched for a patient. The 35-day shelf-life constant is a single classroom rule and differs from the component-specific periods described by the Malaysian dashboard (Ministry of Health Malaysia, 2026a). Compatible mode represents a simplified ABO/Rh rule, not an authorization to transfuse. The software includes no adverse-event or haemovigilance workflow. WHO's description of a safe blood system includes governance, screening, quality systems, and clinical practice that fall outside this project (World Health Organization, 2026a).

Operationally, the application is a local single-user desktop system. A file lock prevents concurrent processes instead of supporting them. Account stores have fewer recovery controls than the main state store. The checksum is intentionally strict but fragile across unversioned schema changes. There is no remote backup, audit identity, database transaction log, monitoring, or automatic update mechanism. These limits make production deployment inappropriate.

7.3 Testing and maintainability limitations

The test suite is broad but its size must not be confused with measured coverage. No line, branch, or mutation coverage report is generated. Some Swing tests inspect component structure and state without automating a complete rendered login-to-logout scenario. Manual visual review remains necessary for spacing, clipping, keyboard behaviour, focus order, contrast, and different display scaling. The current report screenshots confirm the principal layouts on one macOS environment only.

LifeFlowController contains donor management, inventory, requests, matching, decline logic, appointments, identifiers, and commit coordination. Dividing it into focused application services would reduce change risk while keeping one transaction coordinator. The ReportExporter interface should either be implemented by exporter objects or removed in favour of an explicit static utility design. Account persistence should adopt a versioned envelope, hashing, backup, and recovery policy consistent with the main store.

Future work should add password hashing, configurable administrator setup, account-store migration, JaCoCo reporting, accessibility review, and a small GUI smoke suite that exercises the built JAR. Persistence changes should be accompanied by format-version tests and fixtures from older builds. None of these items is described as already implemented. They form a roadmap based on the current gaps.

8. Conclusion

8.1 Conclusion

LifeFlow applies object-oriented programming to a relevant SDG 3 problem domain through a working Java Swing simulation. It represents donors, units, hospitals, requests, fulfilments, and appointments as connected objects; separates request subtypes through inheritance and runtime polymorphism; uses interfaces and generics for reusable contracts; and relies on collections and JSON file handling for operational state. Its processing includes eligibility decisions, priority ordering, exact or compatible matching, FEFO selection, reservations, fulfilment, decline, stale cleanup, and transactional rollback.

The fresh verification result shows that the current code builds cleanly and that 229 Surefire cases complete without failures, errors, or skips. The screenshots and exported reports demonstrate the graphical and file outputs, while representative tests explain how critical behaviours are checked. The evidence also identifies genuine limitations: simplified medical rules, insecure local credentials, single-user storage, a large controller, unequal resilience between stores, and no measured coverage percentage.

The project therefore satisfies the central educational goals of the assignment without claiming clinical or production readiness. Its contribution is a traceable example of how encapsulation, inheritance, polymorphism, abstraction, collections, persistence, and testing can be combined around one coherent real-world simulation. A future version could improve security, modularity, accessibility, and automated GUI verification while preserving the deterministic domain rules established here.

References

- Ministry of Health Malaysia. (2026a). Blood donation. KKMNOW. Retrieved August 24, 2026, from <https://data.moh.gov.my/dashboard/blood-donation>
- Ministry of Health Malaysia. (2026b). Daily blood donations by blood group. KKMNOW. Retrieved August 24, 2026, from https://data.moh.gov.my/data-catalogue/blood_donations
- United Nations. (n.d.). Goal 3: Ensure healthy lives and promote well-being for all at all ages. Retrieved August 24, 2026, from <https://sdgs.un.org/goals/goal3>
- World Health Organization. (2026a, June 12). Blood safety and availability. <https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability>
- World Health Organization. (2026b, June 12). Global status report on blood safety and availability 2025. <https://www.who.int/publications/i/item/9789240121546>

Appendix A : Representative Test Matrix

The representative set is intentionally smaller than the full Surefire result. It concentrates on behaviours that can be discussed clearly during assessment and links each explanation to an existing class and method.

Table 8. Representative test discussion matrix

Behaviour	Test class and method	Expected evidence
Request priority and ties	MatchingConsistencyTest.orderBreaksTiesByRequestDateThenId	Priority, date, then identifier form a deterministic order.
FEFO allocation	ControllerReliabilityTest.matchingUsesFefoAndPersistsTheAuditInOneSnapshot	The earliest-expiring eligible unit is consumed and audit data shares one commit.
Insufficient stock	ControllerReliabilityTest.insufficientStockDoesNotSaveOrChangeRevision	No partial save or revision change occurs.
Save rollback	ControllerReliabilityTest.failedMatchingSaveRollsBackUnitsRequestAndAuditTogether	Injected I/O failure leaves the published snapshot unchanged.
Checksum corruption	JsonLifeFlowStoreTest.rejectsTamperedPrimaryAndRestoresVerifiedBackup	Tampering blocks normal load and recovery uses a verified backup.
Backup generation	JsonLifeFlowStoreTest.fallsBackToPreviousBackupAndMakesItCurrent	The previous verified backup remains recoverable.
Migration	JsonMigrationTest.migratesVersionOneAndRemovesDuplicatedDonorDonationDate	Version 1 data is converted to the current representation.
Appointment booking	AppointmentBookingTest.doubleBookingIsRejected	A donor cannot hold two active bookings.
Volunteer reservation	AppointmentBookingTest.volunteerDonationsReserveUnitsUntilRequestIsComplete	Committed units stay unavailable to unrelated matching.
Request decline	AppointmentBookingTest.decliningRequestReleasesReservedUnitsBackToStock	Cancellation restores committed stock.
Role UI	HospitalAppointmentsUiTest.hospitalSeesItsAppointmentsAndVolunteers	Hospital ownership and volunteer counts are visible.
Report safety	ReportExporterTest.summaryHtmlContainsMetricsAndEscapesMarkup	HTML output includes metrics without rendering injected markup.

Appendix B: Team Contribution Matrix

Table 9. Team contribution matrix

Full Name	Role	Main Responsibilities
ALI MOHAMMED RIYADH YASEEN FAREA	Project Leader and Lead Developer	Defined the overall system direction, designed the architecture and object model, developed most of the core functionality, integrated the project components, resolved major technical issues, reviewed the codebase, and supervised final testing and documentation.
OMAR ALGAAFARI	Business Logic Developer	Contributed to the system's business rules and data processing, including donor eligibility, blood inventory management, request prioritisation, blood-type compatibility, matching, and request fulfilment.

Full Name	Role	Main Responsibilities
KISRANE AMINE	User Interface Developer	Contributed to the Java Swing interfaces for administrators, hospitals, and donors. Worked on navigation, forms, tables, input validation, feedback messages, and presentation of system data.
GHAITH ALDAIN GAMIL HAMID MELHI QASEM	Testing and Quality Assurance Engineer	Tested the main system functions and exceptional cases, reviewed Maven test results, verified integration between the interfaces and business logic, recorded identified problems, and helped confirm that corrections worked as expected.
JIHA OMAR	Research and Documentation Coordinator	Researched SDG 3 and the project background, organised the assignment requirements, contributed to the project report, arranged references, figures, and tables, and reviewed the documentation and submission formatting.